NVIDIA VSP integration — test commands

Date: 2026-10-25. Repo: `github.com/kartikeyg0104/opi-nvidia-dpf-adapter`, branch `LifecycleManager`. Lab host: dh2 `172.22.1.2`, k3s `v1.36.4+k3s1`, DPF `v26.4.0`. This Mac is the VPN Mac. Do not copy `opi_lab_key`. Do not change `/etc`, packages, systemd, firmware, or BMC. Do not run `mlxfwreset`. Do not `helm uninstall` DPF.

1. Reach dh2

```
SSH_OPTS=(
  -o IdentityAgent=none
  -o IdentitiesOnly=yes
  -o IdentityFile="$HOME/.ssh/opi_lab_key"
  -o UseKeychain=yes
  -o KexAlgorithms=curve25519-sha256
  -o ConnectTimeout=20
)
ssh "${SSH_OPTS[@]}" root@172.22.1.2 'hostname; date -u'
```

On dh2, `KUBECONFIG=/etc/rancher/k3s/k3s.yaml`. Job files go only in `/tmp/lab-job-<name>/` and are deleted afterward. Pack the tree with `COPYFILE_DISABLE=1` and `find . -name '._*' -delete`, because AppleDouble files break YAML load. Tests that open `../../config` need a repo-shaped working directory. dh2 has no Go toolchain. Compile on the Mac:

```
cd opi-nvidia-dpf-adapter
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 GOTOOLCHAIN=go1.26.10 \
  go test -c -o /tmp/discovery.test ./pkg/discovery/
```

2. Local unit and integration

```
cd /Users/kartikey0104/Desktop/opi-dpf-adapter/opi-nvidia-dpf-adapter
GOTOOLCHAIN=go1.26.10 go test ./pkg/discovery/ ./pkg/vsp/ \
  ./pkg/mapping/ ./pkg/translation/ ./pkg/lifecycle/ \
  ./cmd/opi-daemon/ ./cmd/vsp/ ./test/conformance/ \
  -count=1 -timeout 15m
```

VSP socket tests need to run outside the sandbox (`required_permissions: all`) because they bind a unix socket.

Focused suites used while building this work:

```
GOTOOLCHAIN=go1.26.10 go test ./pkg/discovery/ -count=1 -timeout 4m \
  -run 'PCI|Annotator|Inventory|Identity|Live'
GOTOOLCHAIN=go1.26.10 go test ./pkg/vsp/ -count=1 -timeout 4m \
  -run 'GetDevices|Handshake|Handoff|NetworkFunction|Socket|Grpcurl'
GOTOOLCHAIN=go1.26.10 go test ./pkg/mapping/ ./pkg/translation/ \
  -count=1 -timeout 4m \
  -run 'Flavor|Nvconfig|DPUReady|ReconcileDPUFlavor'
```

`make test` (envtest assets for Kubernetes 1.36) is the full non-e2e suite. Recorded local result in `evidence/unit-tests.log`: `cmd/vsp`, `pkg/discovery`, `pkg/lifecycle`, `pkg/mapping`, `pkg/translation`, `pkg/vsp`, and `test/conformance` all `ok`.

3. What each package proves

Command	Proves
`go test ./pkg/discovery/`	PCI vendor/product filter, VPD and DSN serial, no `0xc2d3` fallback, permission denied is an error, inventory assign/clear
`go test ./pkg/vsp/`	Both gRPC APIs, socket share, Create/Delete only after a DPUService is accepted
`go test ./cmd/opi-daemon/`	GrpcPlugin client Init and GetDevices
`go test ./pkg/mapping/`	Flavor nvconfig CEL, including current-firmware annotation overrides
`go test ./pkg/translation/`	Reconcile writes the same nvconfig the mapping emits

Command	Proves
`go test ./test/conformance/`	Create without a handoff errors; StaticHandoff then succeeds
`make test-e2e`	Kind cluster, mock VSP, DPU to DPUDevice, SFC to DPUService

4. Socket check with no cluster

Mock serial `MTEXAMPLE0001` at PCI `0000:03:00.0`. macOS cannot bind `/var/run`, so use `/tmp`.

```
go run ./cmd/vsp --grpc-only --serial-number=MTEXAMPLE0001 \
  --grpc-socket=/tmp/vendor-plugin.sock
grpcurl -plaintext unix:///tmp/vendor-plugin.sock list
grpcurl -plaintext -d '{}' unix:///tmp/vendor-plugin.sock \
  Vendor.DeviceService/GetDevices
grpcurl -plaintext -d '{}' unix:///tmp/vendor-plugin.sock \
  opi_api.lifecycle.v1alpha1.DeviceService/GetDevices
grpcurl -plaintext -d '{"dpuMode":true}' unix:///tmp/vendor-plugin.sock \
  Vendor.LifeCycleService/Init
```

`grpc-only` has no cluster client. CreateNetworkFunction and DeleteNetworkFunction must fail (`RejectHandoff`). A socket listing is not DPF acceptance.

Expected services: `Vendor.DeviceService`, `Vendor.HeartbeatService`, `Vendor.LifeCycleService`, `Vendor.NetworkFunctionService`, `opi_api.lifecycle.v1alpha1.DeviceService`, `opi_api.lifecycle.v1alpha1.LifeCycleService`.

Expected GetDevices:

```
{"devices":{"0000:03:00.0":{"ID":"MTEXAMPLE0001","health":"Healthy"}}}
```

5. Live BlueField proof on dh2

Hardware socket (root, mode `srwxr-xr-x`):

```
/var/run/dpu-daemon/vendor-plugin/vendor-plugin.sock
```

Daemon client, one shot, identifier is the VPD serial:

```
opi-daemon --once \
  --grpc-socket=/var/run/dpu-daemon/vendor-plugin/vendor-plugin.sock \
  --dpu-identifier=MT2321XZ0KVX
```

Observed 2026-10-25 08:54:11Z: `Init/GetDevices ok`, devices=1, `pci=0000:0f:00.0`, `id=MT2321XZ0KVX`, `health=Healthy`.

Read the card and the objects:

```
lspci -nn -s 0000:0f:00.0
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
kubectl get dpu -n dpf-operator-system dh2-bluefield-2
kubectl get dpudevice -n dpf-operator-system mt2321xz0kvx
kubectl get dataprocessingunit dh2-bluefield-2
kubectl -n opi-nvidia-dpf-adapter-system get pods
kubectl get dpuservice -A
```

Live ClusterHandoff on dh2: 11 DPUServices with a chart set, `createErr=<nil>`. An empty chart or a deleting DPUService still refuses.

6. Kind end to end

```
make test-e2e
```

This needs Docker and a rebuilt image that contains `/opi-daemon`. On 2026-10-25 Docker was not running (`unix:///Users/kartikey0104/.docker/run/docker.sock` missing) and the data volume had been full. Kind was not completed.

7. Boot status (read only)

```
kubectl -n dpf-operator-system get dpu dh2-bluefield-2
```

Phase `Rebooting`, previous phase `DPU Config`. Agent `RebootMethodDiscovery=True/SystemLevelReset`. `reset_needed=true`. Command required: reboot the external host. Do not run `mlxfwreset` from this adapter.

8. Commands that were not run

```
mlxfwreset -d /dev/mst/mt41686_pciconf0 reset --level 3
helm uninstall dpf-operator -n dpf-operator-system
# do not delete DPU/dh2-bluefield-2
# finalizer provisioning.dpu.nvidia.com/dpu-protection deprovisions
```